SPI-Based Master-Slave Processing System on Basys 3 FPGA

Group Members-V.Jyoshna,K.Pravallika

Objective:-

The core objective of this project is the design and implementation of a robust **Serial Peripheral Interface (SPI)** communication system utilizing the **Basys 3 FPGA** development board. This system serves as a practical demonstration of synchronous serial data transmission between a centralized controller (Master) and peripheral processing units (Slaves).

The architecture is specifically designed to support:

- **Single SPI Master**: Responsible for initiating communication, managing clock signals, and selecting targets.
- **Dual SPI Slaves**: Two independent modules capable of receiving data, performing localized arithmetic or logical operations, and returning status information.
- **User Interface Integration**: Utilizing 8-bit switch inputs for data/command entry and 7-segment displays for real-time output visualization.
- **Feedback Loop**: Implementing a MISO (Master In Slave Out) path to display historical data via onboard LEDs.

System Overview:-

The SPI Master sends an 8-bit data frame to one of the two Slave modules. This 8-bit frame contains information about which slave should be selected, which operation should be performed, and the 4-bit data value to be processed.

When the transfer starts, the Master sends the data bit by bit through the MOSI line and also generates a clock signal to keep communication synchronized. Based on the slave select bit, only one slave becomes active and receives the data, while the other slave stays inactive.

After receiving all 8 bits, the selected Slave reads the command and performs the required operation on the 4-bit input data. These operations can include complement, addition, subtraction, multiplication, reversing bits, or checking parity. The output is then stored inside the slave.

The processed result is displayed on the 7-segment display so the user can directly see the output. At the same time, the slave sends its previously stored value back to the Master through the MISO line. This old value is shown on the LEDs, which helps the user compare the previous result with the new one.

This process shows how SPI allows two-way communication, where the Master can send new data while receiving old data from the Slave at the same time.

8-Bit Frame Format :-

The system receives an 8-bit input from the Basys 3 board switches, represented as SW[7:0]. These 8 switches are used to enter the complete SPI data frame manually before starting communication.

The 8-bit frame is divided into three parts:

- **SW[7]** – Slave Select Bit: Used to choose the active slave. 0 selects Slave 0 and 1 selects Slave 1.
- **SW[6:4]** – Command Bits: These 3 bits define the operation to be performed, such as complement, addition, subtraction, multiplication, or parity check.
- **SW[3:0]** – Data Bits: These 4 bits contain the input data that will be processed by the selected slave.

This structure combines slave selection, command, and data into one 8-bit frame for SPI communication.

Slave Selection:-

- **0 → Slave 0:** When the slave select bit is set to 0, the Master communicates with Slave 0. Only Slave 0 becomes active, receives the input data, performs the required operation, and displays the result.
- **1 → Slave 1:** When the slave select bit is set to 1, the Master communicates with Slave 1. In this case, only Slave 1 is activated to process the received data while Slave 0 remains inactive.

Command Operations:-

- **000 → Pass / Swap Data:** The input 4-bit data is directly stored and displayed without any modification. This is useful for simply transferring and viewing the original data.
- **001 → 1's Complement:** All bits of the input data are inverted, meaning every 0 becomes 1 and every 1 becomes 0.
- **010 → 2's Complement:** The system first performs 1's complement on the input data and then adds 1 to get the final result.
- **011 → Addition:** The 4-bit data is split into two 2-bit values, **bits[3:2]** and **bits[1:0]**, and these two values are added together.
- **100 → Subtraction:** The lower 2 bits are subtracted from the upper 2 bits, i.e., **bits[3:2] - bits[1:0]**.
- **101 → Multiplication:** The two 2-bit values are multiplied, **bits[3:2] × bits[1:0]**, to generate the result.
- **110 → Reverse Data:** The bit order of the 4-bit input is reversed before displaying the output.
- **111 → Parity Check:** The system checks the number of 1s in the input data and generates a parity result based on whether the count is even or odd.

Hardware Mapping on Bassys 3:-

1. **SW[7:0] → 8-bit SPI Frame Input:** The 8 switches on the Basys 3 board are used to manually enter the complete 8-bit SPI input frame, including slave selection, command, and data bits.
2. **BTN_U → Start SPI Transfer:** Pressing the Up button starts the SPI communication process between the Master and the selected Slave.
3. **BTN_C → Reset:** The Center button is used to reset the entire system and clear previously stored values.
4. **AN0 → Displays Slave 0 Result:** The 7-segment display AN0 shows the processed output generated by Slave 0.
5. **AN1 → Displays Slave 1 Result:** The 7-segment display AN1 shows the processed output generated by Slave 1.

6. **LED[3:0] → Previous Slave Data:** The LEDs display the previous value stored in the selected Slave, which is sent back to the Master through the MISO line.

Module Description:-

1. SPI Master

The SPI Master is the main controller of the system and is responsible for managing communication between itself and the two Slave modules. Its main functions include:

1. **Reading the 8-bit Frame:** The Master reads the complete 8-bit input frame from the Basys 3 switches (SW[7:0]).
2. **Selecting the Target Slave:** Based on the most significant bit (MSB / SW[7]), the Master decides whether to communicate with Slave 0 or Slave 1.
3. **Serial Data Transmission:** The Master sends the 8-bit frame one bit at a time through the MOSI (Master Out Slave In) line while generating the SPI clock signal.
4. **Receiving Data through MISO:** During the same transfer, the Master receives the previous stored value from the selected Slave through the MISO (Master In Slave Out) line.
5. **Displaying Previous Data:** The received previous value is displayed on the onboard LEDs for user observation.

Implementation Correction:
During development, an issue was found where the Master ended the SPI transaction too early. Because of this, the final SPI clock pulse did not properly reach the Slave, causing the last bit of the 8-bit frame to be missed. After correcting the final clock timing, the Slave was able to receive the complete 8-bit frame successfully, ensuring reliable communication.

2. SPI Slave

- Receives serial data from the master through the **MOSI line** when selected using the **chip select (CS)** signal.
- Operates synchronously with the **clock signal** provided by the master.
- Stores incoming bits in a **shift register** until a complete data frame is received.
- Extracts **command and data fields** from the received frame (based on predefined bit positions).
- Performs the required **internal operation** such as data processing or register update.
- Stores the **new result** in an internal register or memory location.
- Sends the **previous stored value** back to the master through the **MISO line** during the same transaction.

3. 7-Segment Decoder

- Converts **4-bit binary input** into corresponding **7-segment display signals**.
- Acts as an interface between digital data and visual display hardware.
- Decodes inputs from **0 to 15 (hex range)** for display output.

Display Mapping:

- 0–9 → Displays digits **0, 1, 2, 3, 4, 5, 6, 7, 8, 9**
- 10–15 → Displays characters **A, b, C, d, E, F** (where:
    - 10 = A
    - 11 = b
    - 12 = C
    - 13 = d
    - 14 = E
    - 15 = F)
- Each output drives specific **segments (a to g)** of the 7-segment display.
- Ensures proper visual representation of all binary/hex values on the display.

4. Top Module

- Instantiates a single **SPI master** module for controlling communication.
- Instantiates **two SPI slave** modules to act as independent devices.
- Connects shared SPI lines: **MISO, MOSI, SCLK, and SS (chip select)** between master and slaves.
- Manages proper communication routing between master and selected slave.
- **Multiplexes the 7-segment display** for efficient display control.
- Routes output of **Slave 1 to AN0** and **Slave 2 to AN1**.
- Displays **master received data on LEDs** for visual monitoring.

Working Principle:-

- The user sets an **8-bit input frame** using switches on the board.
- On pressing **BTN_U**, the SPI master initiates communication.
- The master selects the appropriate slave using the **slave select (SS)** line and transmits the 8-bit frame.
- The selected slave decodes the frame into **command bits and data bits**.
- Based on the decoded command, the slave performs the required operation (e.g., arithmetic, logic, or data manipulation).
- The computed result is displayed on the corresponding **7-segment display (AN0 or AN1)**.
- The slave's previously stored value is sent back to the master and displayed on the **LEDs**.

Example

- Input: `00001111`
    - Slave select = **0** → **Slave 0**
    - Command = `000`
    - Data = `1111`
    - Slave 0 stores **F** and displays **F on AN0**
- Next input to same slave: `00000101`
    - Slave 0 updates and displays **5**
    - LEDs show previous stored value **F**

Testing and Verification

The system was tested on Basys 3 FPGA using different switch combinations covering all implemented commands.

Sample Test Cases:

- `00001111` → F
- `00010101` → 1's complement of 5 = A
- `00100101` → 2's complement of 5 = B
- `00111011` → 2 + 3 = 5
- `01001110` → 3 − 2 = 1
- `01010110` → 1 × 2 = 2
- `01101100` → reverse 1100 = 0011
- `01111011` → parity of 1011 = 1

Challenges Faced:-

During development of the SPI-based system, several technical issues were encountered:

- Incorrect **SPI transfer completion timing**, causing incomplete or premature transactions.
- **Slave not receiving the final bit** due to clock edge synchronization issues.
- Debugging difficulties in both **7-segment display output and SPI timing signals**.
- **Verilog synthesis-related structural issues**, affecting proper hardware implementation.
- FPGA hardware verification required **step-by-step signal-level debugging** to trace data flow.

Solutions Implemented:-

These issues were resolved through the following improvements:

1. Simplified the **SPI timing logic** to ensure stable data transfer.
2. Corrected the master's behavior to properly generate the **final clock edge**.
3. Used **LED indicators for debugging** intermediate signals and data flow.
4. Verified the **raw data path first**, before integrating full command-processing logic.

Conclusion:-

This project successfully demonstrates **SPI communication between one master and two slaves** implemented on the **Basys 3 FPGA board**. The designed system operates reliably and achieves the intended functionality.

- It correctly **selects one of the two SPI slaves** based on the input frame.
- The selected slave **decodes command bits and performs the required operations**.
- The processed output is displayed on the corresponding **7-segment display**.
- Each slave **returns its previously stored value to the master** during communication.
- The returned data is successfully displayed on the **LEDs of the FPGA board**.

Overall, the project provides a practical demonstration of **SPI-based digital communication**, **serial data transfer**, **arithmetic and logic processing**, and **real-time FPGA hardware debugging and verification**.